

Singleton Pattern: Serialization Attack & Prevention

Overview

This documentation explains how serialization can break the Singleton design pattern in Java and demonstrates the solution using the `readResolve()` method. The Singleton pattern ensures only one instance of a class exists, but serialization/deserialization can inadvertently create multiple instances.

The Problem: Serialization Attack on Singleton

When a Singleton object is serialized and then deserialized, Java's default deserialization mechanism creates a **new instance** of the object, bypassing the private constructor and breaking the Singleton guarantee.

Example Code

```
java
```

```
package org.systemDesign.creationtionalPattern.singleton.failure;
```

```
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.IOException;  
import java.io.ObjectInputStream;  
import java.io.ObjectOutputStream;  
import java.io.Serial;  
import java.io.Serializable;  
import java.util.Objects;
```

```
class SerializeSingleton implements Serializable {  
    @Serial  
    private static final long serialVersionUID = 1L;  
    private static volatile SerializeSingleton instance;  
    private final String appName;  
    private final String version;  
  
    private SerializeSingleton(){  
        appName = "MyApp";  
        version = "1.0";  
    }  
  
    public static SerializeSingleton getInstance(){  
        if(Objects.isNull(instance)){  
            synchronized (SerializeSingleton.class){  
                if(Objects.isNull(instance)){  
                    instance = new SerializeSingleton();  
                }  
            }  
        }  
        return instance;  
    }  
  
    public String getAppName() {  
        return appName;  
    }  
  
    public String getVersion() {  
        return version;  
    }  
}
```

```
//To prevent this attack, add readResolve():
```

```

@Serial
protected Object readResolve() {
    return getInstance(); // Return existing singleton
}
}

public class SerializationAttack {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        // Get first instance normally
        SerializeSingleton instance1 = SerializeSingleton.getInstance();

        //Serialize
        /*
        Creates a FileOutputStream that opens/creates file singleton.ser in binary write mode
        Wraps it with ObjectOutputStream which converts Java objects to byte stream
        */
        ObjectOutputStream out = new ObjectOutputStream(
            new FileOutputStream("singleton.ser"));
        /*
        Serialization happens here
        JVM inspects instance1 and writes:
        Class metadata (class name, serialVersionUID)
        Field names and types
        Field values (appName = "MyApp", version = "1.0")
        static fields (instance) are NOT serialized (they belong to class, not object)
        */
        out.writeObject(instance1);
        //Flushes remaining bytes to file and releases resources
        out.close();

        //Deserialize - create NEW instance
        /*
        Opens singleton.ser in binary read mode
        Wraps with ObjectInputStream to convert byte stream back to objects
        */
        ObjectInputStream in = new ObjectInputStream(
            new FileInputStream("singleton.ser"));

        /*
        Deserialization happens here
        JVM does the following:
        Reads class metadata from file
        Allocates memory for NEW object (bypasses constructor!)
        Reads field values and assigns them directly to the new object

```

Checks if readResolve() exists → if yes, returns its result instead

*/

```
SerializeSingleton instance2 = (SerializeSingleton) in.readObject();
```

//Releases file resources

```
in.close();
```

/*

Why Singleton Breaks (Without readResolve)

Step	What Happens
------	--------------

writeObject()	Saves object state to bytes
---------------	-----------------------------

readObject()	Creates brand new object in memory
--------------	------------------------------------

Result	instance1 != instance2 (two different objects)
--------	--

How readResolve() Fixes It

@Serial

```
protected Object readResolve() {
```

```
    return getInstance(); // Return existing singleton
```

```
}
```

After readObject() reconstructs the new object, JVM checks for readResolve():

If present → discards the newly created object

Returns getInstance() instead → the original singleton

Result: instance1 == instance2 is true

*/

```
System.out.println(instance1 == instance2); // false - Singleton broken by Serialization/Deserialization!
```

```
        // true - After handling it with readResolve
```

```
}
```

```
}
```

Serialization & Deserialization: Step-by-Step Data Conversion

Serialization: Object → Bytes

Step 1: Object in Memory (Before Serialization)

SerializeSingleton object in heap:

```
| appName = "MyApp" (String ref) |
```

```
| version = "1.0" (String ref) |
```

Step 2: `writeObject()` converts to byte stream

Binary Stream Structure (simplified):

STREAM_MAGIC (0xACED) - Identifies as Java serialization	
STREAM_VERSION (0x0005) - Protocol version	
TC_OBJECT (0x73) - "This is an object"	
CLASS DESCRIPTOR:	
- Class name: "org.systemDesign...SerializeSingleton"	
- serialVersionUID: 1L	
- Field count: 2	
- Field 1: String appName	
- Field 2: String version	
FIELD VALUES:	
- appName → TC_STRING → length(5) → "MyApp"	
- version → TC_STRING → length(3) → "1.0"	

Step 3: Written to file as raw bytes

File: singleton.ser (binary)

AC ED 00 05 73 72 00 3A 6F 72 67 2E 73 79 73 74

65 6D 44 65 73 69 67 6E ... (continues)

Deserialization: Bytes → Object

Step 1: Read bytes from file

java

```
ObjectInputStream in = new ObjectInputStream(new FileInputStream("singleton.ser"));
```

Step 2: Parse stream header

Read 0xACED → Valid Java serialization stream ✓

Read 0x0005 → Protocol version 5 ✓

Step 3: Read class descriptor

Read class name → "org.systemDesign...SerializeSingleton"

Find class in classpath → Found ✓

Compare serialVersionUID → Matches (1L) ✓

Step 4: Allocate memory WITHOUT calling constructor

java

// JVM internally does something like:

`SerializeSingleton obj = Unsafe.allocateInstance(SerializeSingleton.class);`

// Constructor is NEVER called!

Step 5: Read and assign field values directly

Read "MyApp" → `obj.appName = "MyApp"`

Read "1.0" → `obj.version = "1.0"`

Step 6: Check for `readResolve()`

java

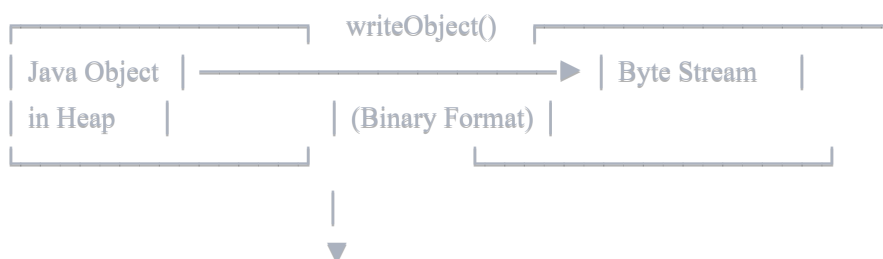
// JVM checks: Does class have readResolve()?

```
if (hasReadResolve) {  
    return obj.readResolve(); // Returns getInstance()  
} else {  
    return obj; // Returns the NEW object (singleton broken!)  
}
```

Visual Summary

Serialization Flow

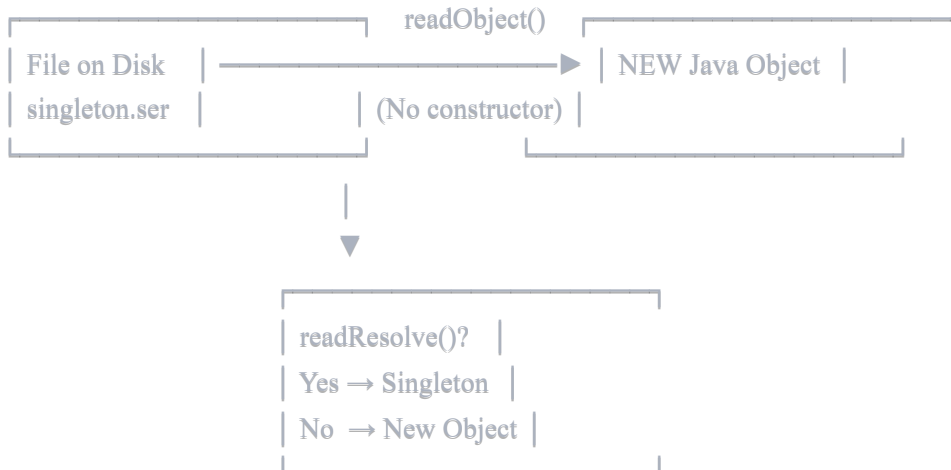
SERIALIZATION:



```
File on Disk |
singleton.ser |
```

Deserialization Flow

DESERIALIZATION:



Why Singleton Breaks (Without `readResolve()`)

Step	What Happens
<code>writeObject()</code>	Saves object state to bytes
<code>readObject()</code>	Creates brand new object in memory
Result	<code>instance1 != instance2</code> (two different objects)

How `readResolve()` Fixes It

```
java
```

```
@Serial
protected Object readResolve() {
    return getInstance(); // Return existing singleton
}
```

What Happens:

1. After `readObject()` reconstructs the new object, JVM checks for `readResolve()`
 2. **If present** → discards the newly created object
 3. Returns `getInstance()` instead → the original singleton
 4. **Result:** `instance1 == instance2` is `true`
-

Key Takeaways

- **Static fields are NOT serialized** (they belong to the class, not the object)
 - Deserialization **bypasses the constructor** completely
 - Without `readResolve()`, deserialization creates a **new instance**, breaking Singleton
 - The `readResolve()` method ensures the existing Singleton instance is returned
 - Always implement `readResolve()` in Serializable Singleton classes to prevent this attack
-

Solution Summary

To protect your Singleton from serialization attacks:

1. Implement `Serializable` interface
2. Add `serialVersionUID` for version control
3. **Implement `readResolve()` method** that returns `getInstance()`

This ensures that even after deserialization, only one instance of the Singleton exists in the JVM.